

CCZ and EA equivalence algorithms

First Author^{1,2*} and Second Author^{2,3†}

^{1*}Department, Organization, Street, City, 100190, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;

Contributing authors: iauthor@gmail.com;

[†]These authors contributed equally to this work.

Abstract

The abstract serves both as a general introduction to the topic and as a brief, non-technical summary of the main results and their implications. Authors are advised to check the author instructions for the journal they are submitting to for word limits and if structural elements like subheadings, citations, or equations are permitted.

Keywords: keyword1, Keyword2, Keyword3, Keyword4

1 Introduction

CCZ- and EA-equivalence are two standard notions for comparing vectorial Boolean functions $F, G : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ up to transformations that preserve many cryptographic properties. We refer to [1] for background and for a detailed discussion of the relationship between these notions.

1.1 Equivalence relations

The *graph* of F is the set

$$\Gamma_F = \{(x, F(x)) : x \in \mathbb{F}_2^n\} \subseteq \mathbb{F}_2^{n+m}.$$

We say that F and G are *CCZ-equivalent* if there exists an affine permutation $A : \mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m}$ such that

$$A(\Gamma_G) = \Gamma_F.$$

Equivalently, CCZ-equivalence is affine equivalence of the two graphs viewed as subsets of $\mathbb{F}_2^{n+m}$.

We say that F and G are *EA-equivalent* if there exist affine permutations $A_1 : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ and $A_2 : \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$, and an affine map such that. $A_3 : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$

$$F(x) = A_2(G(A_1(x))) + A_3(x)$$

EA equivalence is a restricted version of CCZ-equivalence, where the affine map has the following form

$$A(x, y) = (L_1(x) + a, L_2(y) + L_3(x) + b),$$

where L_1, L_2 are linear permutations, L_3 a linear map, and a, b constants.

2 Previous work

2.1 Invariants

2.2 Code equivalence

For a general function F the state of the art for testing CCZ/EA-equivalence is to reduce the problem to code equivalence.

Let $\mathcal{C}, \mathcal{D} \subseteq \mathbb{F}_2^N$ be linear codes of the same length N . We say that $\mathcal{C}$ and $\mathcal{D}$ are *permutation-equivalent*, written $\mathcal{C} \sim_{\text{perm}} \mathcal{D}$, if and only if there exists a permutation $\pi \in S_N$ such that

$$\mathcal{D} = \{(c_{\pi(1)}, \dots, c_{\pi(N)}) : (c_1, \dots, c_N) \in \mathcal{C}\}.$$

For any $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. define the binary linear code $\mathcal{C}_1(F)$ of length 2^n by

$$\mathcal{C}_1(F) = \left\{ (a \cdot x + b \cdot F(x) + c)_{x \in \mathbb{F}_2^n} : a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m, c \in \mathbb{F}_2 \right\} \subseteq \mathbb{F}_2^{2^n}.$$

Then we have [2]

$$F \sim_{\text{CCZ}} G \iff \mathcal{C}_1(F) \sim_{\text{perm}} \mathcal{C}_1(G).$$

For EA equivalence we define a similar code

$$\mathcal{C}_2(F) = \left\{ \begin{pmatrix} a \cdot x + b \cdot F(x) + c \\ (b \cdot y)_{y \in \mathbb{F}_2^m \setminus \{0\}} \end{pmatrix}_{x \in \mathbb{F}_2^n} : a \in \mathbb{F}_2^n, b \in \mathbb{F}_2^m, c \in \mathbb{F}_2 \right\} \subseteq \mathbb{F}_2^{2^n + (2^m - 1)}.$$

and we then get that

$$F \sim_{\text{EA}} G \iff \mathcal{C}_2(F) \sim_{\text{perm}} \mathcal{C}_2(G).$$

Having reduced the problem to Code equivalence we can then use a standard algorithm for this well studied problem. Magma seems to have the best implemented algorithm to solve this problem, and although the exact details of their algorithm is not public, they implemented a version of the partition backtracking algorithm introduced in [3].

2.3 Specialized algorithms

For CCZ-equivalence no specialized algorithms have been developed, for EA-equivalence there are a few published algorithms.

For quadratic functions there is the algorithm in [4]. Quadratic functions are quite important since in the case of quadratic apn functions EA and CCZ equivalence coincides [5]. The worst case for the algorithm however is apn functions, especially when the functions are not equivalent. In this paper they also introduce the orthoderivative which is a unique function

3 Partition refinement algorithm

The partition refinement framework was introduced in [6] for the graph isomorphism problem, but has now been adapted for various other problems including code isomorphism [3]. The basic structure of the algorithm is a backtracking search algorithm where each search node have one or more ordered partition, restricting the possible extension of the current partial isomorphism.

For any set S an ordered partition of a set S is a ordered tuple

$$\Pi = (B_1, B_2, \dots, B_k)$$

where

1. *Nonempty blocks:* $B_i \neq \emptyset$ for all i .
2. *Pairwise disjointness:* $B_i \cap B_j = \emptyset$ for all $i \neq j$.
3. *Cover:* $\bigcup_{i=1}^k B_i = S$.

The partitions are ordered so two partitions are consider different if they have the same blocks but in different order. We say that two partitions $\Pi = (B_1, \dots, B_k)$ and $\Pi' = (C_1, \dots, C_l)$ have the same shape if $k = l$ and $|B_i| = |C_i|$ for all i . We will denote singleton cells by I

$$I(\Pi) = \{P \in \Pi : |P| = 1\}$$

In our

In our case the partial isomorphism is the partial affine map $A : D \subseteq \mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m}$. We will denote $\hat{A}$ as an extension of A if $\hat{A}$ agrees with A on D .

We will maintain two ordered partitions, Λ_F and Λ_G , of Γ_F and Γ_G respectively, maintaining

Invariant property 1 For the partial affine map A , any extension $\hat{A}$ satisfies $\hat{A}(P_i^F) = P_i^G$ for all i .

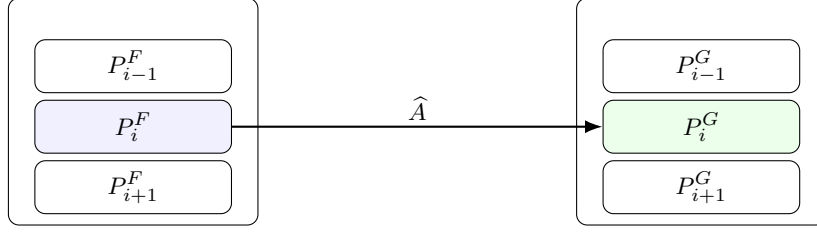


Fig. 1 Visualization of Invariant property 1: any extension $\hat{A}$ of the current partial affine map must map each cell P_i^F to the corresponding cell P_i^G .

This property allows so reduce the number of guesses we have to make for a domain point, since it can only be mapped to a point in the corresponding cell. We then of course want the cells to be as small as possible. To reduce the size of the cells we will refine the point partition, we say that $\hat{\Pi}$ is a refinement of Π if every cell of $\hat{\Pi}$ is contained in some cell of Π , i.e.,

$$\hat{P} \in \hat{\Pi} \implies \exists P \in \Pi \text{ with } \hat{P} \subseteq P.$$

and $\hat{\Pi}$ still satisfies [Invariant property 1](#). If a cell $\hat{P} \in \hat{\Pi}$ have $\hat{P} \subsetneq P$ (so strict subset) we say that $\hat{P}$ have been split.

The basic sructure of the algorithm looks like this:

Algorithm 1: BACKTRACKSEARCH(A, Π_F, Π_G)

```

REFINE( $\Pi_F, \Pi_G$ );
Let  $P_i^F$  be the smallest cell of  $\Pi_F$ ;
Let  $x$  be any element of  $P_i^F$ ;
for  $y \in P_i^G$  do
     $A(x) \leftarrow y$ ;
    BACKTRACKSEARCH( $A, \Pi_F, \Pi_G$ );

```

3.1 Point partition

Lets now move into the spesific details of our algorithm. For our algorithm we will keep an ordered partition of the points in Γ_F and Γ_G for each function, and we will denote these by

$$\Pi_F = (P_1^F, \dots, P_k^F), \quad \Pi_G = (P_1^G, \dots, P_k^G),$$

respectively.

We will enforce that these ordered partitions satisfies [Invariant property 1](#) so thats why we can assume the partitions of Γ_F and Γ_G have the same shape.

Initially the ordered point partitions will have a single cell, simply $\Pi_F = (\Gamma_F)$ and $\Pi_G = (\Gamma_G)$. Since any CCZ mapping A must satisfy $A(\Gamma_F) = \Gamma_G$ by definition, we see that these partitions satisfy [Invariant property 1](#)

3.2 Hyperplane partition

We will keep another ordered partition for each function, whose elements are affine hyperplanes. So our ordered partitions are on the set $\text{AGr}(n + m - 1, n + m)$

$$\Lambda_F = (S_1^F, \dots, S_l^F), \quad \Lambda_G = (S_1^G, \dots, S_l^G).$$

In the same way as the point partition the algorithm will maintain the following invariant for the

Invariant property 2 For the partial affine map A , any extension $\hat{A}$ satisfies $\hat{A}(S_i^F) = S_i^G$ for all i .

We will initialize these partitions differently for EA and CCZ equivalence.

In the case of CCZ equivalence we will set $\Lambda_F = \Lambda_G = (\text{AGr}(n + m - 1, n + m))$, so just a single cell containing all affine hyperplanes of $\mathbb{F}_2^{n+m}$. This of course satisfies [Invariant property 2](#) as any affine map A maps an affine subspace to another affine subspace.

For EA equivalence we can do slightly better (more refined) with the initial partition.

Let

$$H_{\alpha,c} = \{(x, y) : \alpha \cdot x = c\} \subseteq \mathbb{F}_2^{n+m}$$

then we define the set of these hyperplanes as

$$\mathcal{H} = \{H_{\alpha,c} : \alpha \in \mathbb{F}_2^n \setminus \{0\}, c \in \mathbb{F}_2\}$$

EA mappings A preserve these hyperplanes.

Lemma 1 If A is an EA mapping then

$$A(\mathcal{L}) = \mathcal{L}$$

so $A(H) \in \mathcal{H}$ for any $H \in \mathcal{H}$.

Proof We can write the mapping A as $A(x, y) = (L_1(x) + a, L_2(y) + L_3(x) + b)$ where L_1, L_2 are permutations and L_3 is a linear map. Let

$$\alpha' = L_1^{-T}(\alpha), \quad c' = c + \alpha \cdot L_1^{-1}a$$

We then want to show that $A(L_{\alpha,c}) = L_{\alpha',c'}$. Pick some $(x, y) \in L_{\alpha,c}$ and let $(x', y') = A(x, y)$.

$$A(x, y) = (L_1(x) + a, L_2(y) + L_3(x) + b)$$

$L_{\alpha',c'} = \{(x, y) : x \cdot \alpha' = c'\}$ putting (x', y') into this equation we get

$$\begin{aligned} \alpha' \cdot x' &= \alpha'(L_1(x) + a) \\ &= (L_1^{-T}(\alpha))(L_1(x) + a) \end{aligned}$$

$$\begin{aligned}
&= L_1^{-T}(\alpha)L_1(x) + (L_1^{-T}(\alpha)) \cdot a \\
&= \alpha \cdot L_1^{-1}(L_1(x)) + L_1^{-T}(\alpha) \cdot a \\
&= \alpha \cdot x + \alpha \cdot L_1^{-1}(a) \\
&= c + \alpha \cdot L_1^{-1}(a) \\
&= c'
\end{aligned}$$

□

This allows a finer initial partition for the hyperplane when testing EA equivalence.

$$\Lambda_F, \Lambda_G = (\mathcal{L}, \text{AGr}(n + m - 1, n + m) \setminus \mathcal{L})$$

The [Invariant property 2](#) then holds for any EA extension of A due to [Lemma 1](#).

3.3 Signatures

Now for the main idea of the algorithm, partition refinement. The basic idea is refinement where we take either a cell in the point partition or hyperplane partition and try to split it. We do this by defining signatures of points and hyperplanes.

A *signature* is simply a function assigning an invariant to each object. Concretely, for a set of objects Ω , a signature is a map

$$\text{sig} : \Omega \rightarrow \Sigma$$

into some set Σ (typically tuples of integers).

Valid signatures. Let A be a partial affine map. We say the pair $(\text{sig}_A, \text{sig}_B)$ is *valid* if, for every extension $\hat{A}$ of A ,

$$\hat{A}(p) = q \implies \text{sig}_A(p) = \text{sig}_B(q).$$

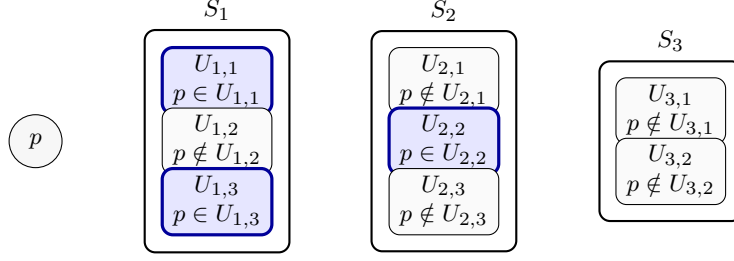
In our case we will define signatures on the points and on the hyperplanes.

3.3.1 Point cells

Lets first consider how we define the signature on point cells. Fix an ordered hyperplane partition $\Lambda = (S_1, \dots, S_l)$. For a point $p \in \Gamma_F$, define its signature as the vector of incidence counts

$$\text{sig}_\Lambda(p) := (|\{U \in S_1 : p \in U\}|, \dots, |\{U \in S_l : p \in U\}|).$$

In other words, the i -th coordinate records how many subspaces in S_i contain p ; crucially, these counts are preserved under CCZ/EA equivalence.



$$\text{sig}_\Lambda(p) = (2, 1, 0) \text{ (illustration)}$$

Fig. 2 point signature: each block S_i contains subspaces U , and the i -th coordinate counts those subspaces with $p \in U$.

Lemma 2 If we have two hyperplane partitions Λ_F and Λ_G satisfying [Invariant property 2](#) for the partial affine map A then $(\text{sig}_{\Lambda_F}, \text{sig}_{\Lambda_G})$ is a valid signature.

Proof Let $\hat{A}$ be any extension of A , and suppose $\hat{A}(p) = q$. Fix $i \in \{1, \dots, l\}$. By [Invariant property 2](#) the extension $\hat{A}$ maps the i -th block of the hyperplane partition to the corresponding block, i.e. $\hat{A}(S_i^F) = S_i^G$.

Consider the map $U \mapsto \hat{A}(U)$, which is a bijection from S_i^F to S_i^G . Moreover,

$$p \in U \iff q = \hat{A}(p) \in \hat{A}(U).$$

Therefore the number of subspaces in S_i^F containing p equals the number of subspaces in S_i^G containing q , i.e.

$$|\{U \in S_i^F : p \in U\}| = |\{V \in S_i^G : q \in V\}|.$$

Since this holds for each i , we get $\text{sig}_{\Lambda_F}(p) = \text{sig}_{\Lambda_G}(q)$, so $(\text{sig}_{\Lambda_F}, \text{sig}_{\Lambda_G})$ is valid. $\square$

3.3.2 Hyperplane signatures

For any affine hyperplane U and a point partition $\Pi = (P_1, \dots, P_k)$ we define the signature of U as

$$\text{sig}_\Pi(U) = (|U \cap P_1|, |U \cap P_2|, \dots, |U \cap P_k|)$$

So the signature of the subspace is the intersection count with the point cells.

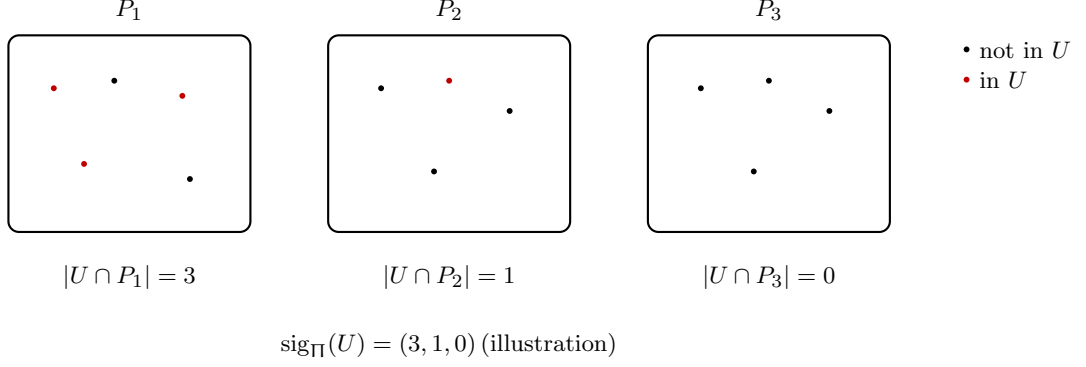


Fig. 3 hyperplane signature: the i -th coordinate is the number of points of P_i that lie in U .

Lemma 3 If we have point partitions Π_F and Π_G satisfying [Invariant property 1](#) for the partial affine map A , then the pair of hyperplane signatures $(\text{sig}_{\Pi_F}, \text{sig}_{\Pi_G})$ is valid.

Proof Let $\hat{A}$ be any extension of A , and suppose $\hat{A}(U) = V$. Fix $i \in \{1, \dots, k\}$. By [Invariant property 1](#) we have $\hat{A}(P_i^F) = P_i^G$. Using that $\hat{A}$ is a bijection, we compute

$$\begin{aligned}
 |V \cap P_i^G| &= |\hat{A}(U) \cap \hat{A}(P_i^F)| \\
 &= |\hat{A}(U \cap P_i^F)| \\
 &= |U \cap P_i^F|.
 \end{aligned}$$

Thus the i -th coordinates of $\text{sig}_{\Pi_F}(U)$ and $\text{sig}_{\Pi_G}(V)$ agree for every i , so $\text{sig}_{\Pi_F}(U) = \text{sig}_{\Pi_G}(V)$. This proves that $(\text{sig}_{\Pi_F}, \text{sig}_{\Pi_G})$ is valid. $\square$

3.4 Going back and forward

We alternate between refining the point partitions and refining the hyperplane partitions, using the fact that each refinement step can create new signatures on the other side.

From hyperplanes to points. Given hyperplane partitions Λ_F and Λ_G , we compute the induced point signatures sig_{Λ_F} and sig_{Λ_G} and refine the point partitions Π_F and Π_G by splitting points that receive different signatures.

From points to hyperplanes. Given point partitions Π_F and Π_G , we compute the induced hyperplane signatures sig_{Π_F} and sig_{Π_G} and refine the hyperplane partitions Λ_F and Λ_G by splitting hyperplanes that receive different signatures.

We repeat these two steps until both (Π_F, Π_G) and (Λ_F, Λ_G) stabilize. After each stabilization step we compare the shapes of the corresponding partitions on the F - and G -sides; if the shapes differ, the branch aborts.

4 Computing partition refinements and signatures

Now let's consider how to actually compute these signatures efficiently.

For a function $f : \mathbb{F}_2^d \rightarrow \mathbb{Z}$ the Fourier transform is defined by

$$\widehat{f}(u) = \sum_{x \in \mathbb{F}_2^d} f(x)(-1)^{u \cdot x}$$

It is well known one can compute this in time $\mathcal{O}(N \log N)$ where N is the size of the domain of f , using the Fast Fourier Transform. In our case the domain is $\mathbb{F}_2^{n+m}$ has size 2^{n+m} so we can do it in time $\mathcal{O}(2^{n+m}(n+m))$.

Another detail is that the algorithm will not keep/work with all hyperplanes. So we will define a set of active cells which are the cells we work with when we refine. So will try to remove hyperplane cells, but make sure to keep at least 2^{n+1} of them. When the number of hyperplanes are small, we will stop Fast Walsh Hadamer transform (FWHT) and just compute $\widehat{f}$ directly as it turns out this is faster in practice. We will not explain the FWHT as it is just a standard well known algorithm. Also computing $\widehat{f}(u)$ is computing the sum for support of f .

4.1 Point refinement

Lets say we are given some ordered partition of points and hyperplanes $\Pi = (P_1, \dots, P_k)$ and $\Lambda = (S_1, \dots, S_l)$. Ideally we want to compute the full point signature

$$\text{sig}_\Lambda(p) := (|\{U \in S_1 : p \in U\}|, \dots, |\{U \in S_l : p \in U\}|),$$

but this is quite expensive; instead we will use a “compressed” version.

We assign some integer s_i to each cell S_i . Then we define a weight function $h : \text{AGr}(n+m-1, n+m) \rightarrow \mathbb{Z}$ by

$$h(U) = s_i.$$

The weighted signature of a point p is defined as

$$\text{sig}_h(p) = \sum_{i=1}^l s_i \cdot |\{U \in S_i : p \in U\}|.$$

This weighted version of the signature is weaker than the original, but it can still distinguish points. When we define weight functions for the hyperplane partitions on the F - and G -sides, we must use the same weights for corresponding blocks. Thus, for two hyperplane partitions Λ_F and Λ_G with corresponding weight functions h_F and h_G , we require $h_F(U) = h_G(V)$ whenever $U \in S_i^F$ and $V \in S_i^G$.

Lemma 4 Let $\Lambda_F = (S_1^F, \dots, S_l^F)$ and $\Lambda_G = (S_1^G, \dots, S_l^G)$ be hyperplane partitions satisfying [Invariant property 2](#) for the current partial affine map A . Let h_F and h_G be weight functions that agree on corresponding blocks (i.e. $h_F(U) = h_G(V)$ whenever $U \in S_i^F$ and $V \in S_i^G$). Then the weighted point signatures $(\text{sig}_{h_F}, \text{sig}_{h_G})$ are valid.

Proof Let $\hat{A}$ be any extension of A , and suppose $\hat{A}(p) = q$. Fix $i \in \{1, \dots, l\}$. By [Invariant property 2](#) we have $\hat{A}(S_i^F) = S_i^G$. The map $U \mapsto \hat{A}(U)$ is therefore a bijection from S_i^F to S_i^G , and $p \in U \iff q \in \hat{A}(U)$. Hence $|\{U \in S_i^F : p \in U\}| = |\{V \in S_i^G : q \in V\}|$ for each i . Since h_F and h_G assign the same weight s_i to corresponding blocks, summing these equalities gives $\text{sig}_{h_F}(p) = \text{sig}_{h_G}(q)$, proving validity. $\square$

The advantage of this signature is that it computed more efficiently using the Fourier transform. Let us now show how to do this. For any $\alpha \in F_2^{n+m}$ and weight function h define the difference function

$$d_h(\alpha) = h(L_{\alpha,0}) - h(L_{\alpha,1})$$

and let $d_h(0) = 0$.

We have the following connection

Lemma 5 For a weight function h for a hyperplane partition Λ

$$\hat{d}_h(p) = 2 \cdot \text{sig}_h(p) - K$$

where $K = \sum_{\alpha \in \mathbb{F}_2^{n+m} \setminus \{0\}} (h(L_{\alpha,0}) + h(L_{\alpha,1}))$.

Proof Fix any $p \in \mathbb{F}_2^{n+m}$.

$$\hat{d}_h(p) = \sum_{\alpha \in \mathbb{F}_2^{n+m}} d_h(\alpha) (-1)^{\alpha \cdot p}$$

Let $c = \alpha \cdot p$ then $d_h(\alpha) (-1)^{\alpha \cdot p} = (h(L_{\alpha,0}) - h(L_{\alpha,1})) (-1)^c$.

$$= 2h(L_{\alpha,c}) - (h(L_{\alpha,0}) + h(L_{\alpha,1}))$$

so we get

$$\begin{aligned} \hat{d}_h(p) &= \sum_{\alpha \in \mathbb{F}_2^{n+m}} d_h(\alpha) (-1)^{\alpha \cdot p} \\ &= \sum_{\alpha \in \mathbb{F}_2^{n+m}} (2h(L_{\alpha,c}) - (h(L_{\alpha,0}) + h(L_{\alpha,1}))) \\ &= \sum_{\alpha \in \mathbb{F}_2^{n+m}} 2h(L_{\alpha,c}) - \sum_{\alpha \neq 0} (h(L_{\alpha,0}) + h(L_{\alpha,1})) \\ &= 2 \cdot \sum_{\alpha \in \mathbb{F}_2^{n+m}} h(L_{\alpha,c}) - K \end{aligned}$$

For each nonzero α , the hyperplane $L_{\alpha,\alpha \cdot p}$ is the unique affine hyperplane with normal α that contains p . Hence

$$\sum_{\alpha \neq 0} h(L_{\alpha,\alpha \cdot p}) = \sum_{U: p \in U} h(U) = \text{sig}_h(p).$$

Substituting this into the last line of the computation gives

$$\hat{d}_h(p) = 2 \cdot \sum_{\alpha \neq 0} h(L_{\alpha,\alpha \cdot p}) - K = 2 \cdot \text{sig}_h(p) - K,$$

as claimed. $\square$

The algorithm for refining the points is then

Algorithm 2: REFINEPOINTS(Π, Λ)

Input: Ordered point partition $\Pi = (P_1, \dots, P_k)$ and ordered hyperplane partition $\Lambda = (S_1, \dots, S_l)$

Output: (**Refined**, Π') where **Refined** is true if at least one cell has been split, and Π' is a refinement of Π

Pick integers $s_1, \dots, s_l$;

$h(U) \leftarrow s_i$ if $U \in S_i$;

$h(U) \leftarrow 0$ if $U \notin \bigcup_i S_i$;

$d(\alpha) \leftarrow h(L_{\alpha,0}) - h(L_{\alpha,1})$;

if $|\bigcup_{i=1}^l S_i| > 2^{n+1}$ **then**

 Compute $\hat{d}(p) = \sum_{\alpha} d(\alpha)(-1)^{\alpha \cdot p}$ for all p using FWHT;

else

 Compute $\hat{d}(p)$ directly by summing over the hyperplanes in Λ .

$\Pi' \leftarrow ()$, **Refined** $\leftarrow$ **false**;

for $P_i \in \Pi$ **do**

 Compute the nonempty blocks $B_{t_1}, \dots, B_{t_r}$ where

$B_t := \{p \in P_i : \hat{d}(p) = t\}$;

if $r > 1$ **then**

Refined $\leftarrow$ **true**;

 Append $B_{t_1}, \dots, B_{t_r}$ to Π' ;

return (**Refined**, Π');

Note that here the algorithm works on the partition for one of the functions, but we still keep track of the correspondence between the F - and G -side partitions; that is, we maintain [Invariant property 1](#). This is simply done by matching the cells that correspond to the same signature. We also pick the integers such that the weight functions for the two functions agree.

4.2 Refining hyperplanes

Now we do the same process to refine hyperplanes. Again we are given a point partition $\Pi = (P_1, \dots, P_k)$ and a hyperplane partition $\Lambda = (S_1, \dots, S_l)$.

As in the point-refinement step, we use a compressed signature. Choose integers $s_1, \dots, s_k$ and define a weight function $w : \Gamma_F \rightarrow \mathbb{Z}$ by

$$w(p) := s_i \quad \text{for } p \in P_i.$$

For an affine hyperplane $L_{\alpha,c}$, define its weighted (compressed) hyperplane signature by

$$\text{sig}_w(L_{\alpha,c}) := \sum_{x \in L_{\alpha,c}} w(x).$$

Lemma 6 Let $\Pi_F = (P_1^F, \dots, P_k^F)$ and $\Pi_G = (P_1^G, \dots, P_k^G)$ be point partitions satisfying [Invariant property 1](#) for the current partial affine map A . Let w_F and w_G be weight functions with the same weights on corresponding point cells (as defined above). Then the weighted hyperplane signatures $(\text{sig}_{w_F}, \text{sig}_{w_G})$ are valid.

Proof Let $\hat{A}$ be an extension of A and assume $\hat{A}(U) = V$. By [Invariant property 1](#), for each i we have $\hat{A}(P_i^F) = P_i^G$, hence $\hat{A}$ restricts to a bijection $U \cap P_i^F \rightarrow V \cap P_i^G$.

Let s_i denote the common value of w_F on P_i^F (equivalently, of w_G on P_i^G). Then

$$\sum_{x \in U \cap P_i^F} w_F(x) = s_i |U \cap P_i^F| = s_i |V \cap P_i^G| = \sum_{y \in V \cap P_i^G} w_G(y).$$

Summing over i gives $\text{sig}_{w_F}(U) = \text{sig}_{w_G}(V)$, proving validity. $\square$

Lemma 7

$$\text{sig}_w(L_{\alpha,c}) = \frac{S + (-1)^c \hat{w}(\alpha)}{2}$$

where $S = \sum_x w(x)$.

Proof Fix $\alpha \neq 0$, we see that

$$\text{sig}_w(L_{\alpha,0}) + \text{sig}_w(L_{\alpha,1}) = \sum_x w(x) = S$$

$$\begin{aligned} \hat{w}(\alpha) &= \sum_x w(x) (-1)^{\alpha \cdot x} \\ &= \sum_{x: \alpha \cdot x = 0} w(x) - \sum_{x: \alpha \cdot x = 1} w(x) \\ &= \text{sig}_w(L_{\alpha,0}) - \text{sig}_w(L_{\alpha,1}) \end{aligned}$$

Solving the linear system

$$\begin{cases} \text{sig}_w(L_{\alpha,0}) + \text{sig}_w(L_{\alpha,1}) = S, \\ \text{sig}_w(L_{\alpha,0}) - \text{sig}_w(L_{\alpha,1}) = \hat{w}(\alpha), \end{cases}$$

Gives $\text{sig}_w(L_{\alpha,0}) = \frac{S + \hat{w}(\alpha)}{2}$ and $\text{sig}_w(L_{\alpha,1}) = \frac{S - \hat{w}(\alpha)}{2}$. $\square$

Algorithm 3: REFINESUPERPLANES(Π, Λ)

Input: Ordered point partition $\Pi = (P_1, \dots, P_k)$ and ordered hyperplane partition $\Lambda = (S_1, \dots, S_l)$
Output: ($\text{Refined}, \Lambda'$) where Refined is true if at least one cell has been split, and Λ' is a refinement of Λ

Pick integers $s_1, \dots, s_k$;
 $w(p) \leftarrow s_i$ if $p \in P_i$;
 $w(p) \leftarrow 0$ if $p \notin \bigcup P_i$;
 $S \leftarrow \sum_{x \in \Gamma_F} w(x)$;
if $|\bigcup S_i| > 2^{n+1}$ **then**
 Compute $\hat{w}(\alpha) = \sum_{x \in \mathbb{F}_2^{n+m}} w(x)(-1)^{\alpha \cdot x}$ for all α using FWHT;
else
 for $\alpha \in \{\alpha : L_{\alpha,0} \text{ or } L_{\alpha,1} \text{ is contained in } \bigcup S_i\}$ **do**
 Compute $\hat{w}(\alpha)$ directly;
 $\Lambda' \leftarrow ()$, $\text{Refined} \leftarrow \text{false}$;
for $S_i \in \Lambda$ **do**
 $\text{sig}_w(L_{\alpha,c}) \leftarrow \frac{S + (-1)^c \hat{w}(\alpha)}{2}$ for all $L_{\alpha,c} \in S_i$;
 Compute the nonempty blocks $B_{t_1}, \dots, B_{t_r}$ where
 $B_t := \{U \in S_i : \text{sig}_w(U) = t\}$;
 if $r > 1$ **then**
 $\text{Refined} \leftarrow \text{true}$;
 Append $B_{t_1}, \dots, B_{t_r}$ to Λ' ;
return ($\text{Refined}, \Lambda'$);

In the same way as in the point partition algorithm, we need to keep track of correspondence between the hyperplane partition on the F and G side, so we maintain [Invariant property 2](#). We do this by picking the same integer for the corresponding cells (so the weight functions agree)

4.3 Refine algorithm

Now for the back and forward refinement algorithm. We now have point partitions Π_F, Π_G and hyperplane partitions Λ_F, Λ_G . We go back and forward between refining the point/hyperplane partitions until no more refinement occurs. Also if a point cell becomes a singleton (and therefore fully determined) we update A and stop if A is fully determined.

As we have explained the way we refine the partitions are through signatures, after refining either the point or hyperplane partitions, all the elements in the same cell will have gotten the same signature in the last iteration of REFINESUPERPLANES or REFINESUPERPLANES (in all iterations actually), so we can assign a signature of each of the cells, denote this value by $\text{sig}(P_i)$ and $\text{sig}(S_i)$ for the points cells and hyperplanes

cells respectively. So $sig(P_i)$ is the signatures all the points got in the last iteration of REFINEPOINTS that was used to create the cell P_i .

Algorithm 4: REFINE($\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$)

Input: Ordered point partitions Π_F, Π_G and ordered hyperplane partitions Λ_F, Λ_G

Output: (IsConsistent, $\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$) after iterated refinement (or false if shapes mismatch)

Refining $\leftarrow$ true;

while Refining **do**

 (RefinedPoints, Π_F) $\leftarrow$ REFINEPOINTS(Π_F, Λ_F);

 (Π_G) $\leftarrow$ REFINEPOINTS(Π_G, Λ_G);

if $|P_i^F| \neq |P_i^G|$ or $sig(P_i^F) \neq sig(P_i^G)$ for any i **then**
 | **return** false

 (RefinedHyperplanes, Λ_F) $\leftarrow$ REFINEHYPERPLANES(Π_F, Λ_G);

 (Λ_G) $\leftarrow$ REFINEHYPERPLANES(Π_G, Λ_G);

if $|S_i^F| \neq |S_i^G|$ or $sig(S_i^F) \neq sig(S_i^G)$ for any i **then**
 | **return** false

 Refining $\leftarrow$ RefinedPoints or RefinedHyperplanes;

Order the partitions Λ_F and Λ_G by size (increasing);

$\Lambda'_F \leftarrow ()$;

$\Lambda'_G \leftarrow ()$;

NumHyperplanes $\leftarrow$ 0;

$i \leftarrow$ 0;

while NumHyperplanes $\leq 2^{n+1}$ **do**

$\Lambda'_F \leftarrow (\Lambda'_F, S_i^F)$;

$\Lambda'_G \leftarrow (\Lambda'_G, S_i^G)$;

 NumHyperplanes $\leftarrow$ NumHyperplanes + $|S_i^F|$;

$i \leftarrow i + 1$;

return (true, $\Pi_F, \Pi_G, \Lambda'_F, \Lambda'_G$);

5 Automorphism groups

Another thing we will take advantage of is the automorphism group of the functions. The CCZ automorphism group consist of all affine permutations such that $A(\Gamma_F) = \Gamma_F$, the EA auto group is the same but A is restricted to EA mappings. We denote these groups by $Aut_{CCZ}(F)$ and $Aut_{EA}(F)$. In this section we are going to deal with both CCZ and EA equivalence and we will just write $Aut(F)$ which will mean either $Aut_{CCZ}(F)$ and $Aut_{EA}(F)$.

Lets say we know a subgroup of the automorphisms groups of G , denote this by H so $H \subseteq Aut(G)$. At a certain point in the algorithm we will be interested in the subgroup of $Aut(G)$ fixing the elements of the domain of the current partial map, let

$$\text{Stab}_A(H) = \{g \in H : g(x) = x \quad \forall x \in \text{Dom}(A)\}$$

so we are talking about what is known the point-wise stabilizer.

Computing the automorphism groups can of course be useful in many applications, we will use to to drastically speed up our CCZ/EA algorithms for symmetric functions (functions with large automorphism groups). If the function is symmetric then we can significantly reduce the branching even if we don't know the entire automorphism group, this is due to the following fact.

Lemma 8 Let $H \leq \text{Aut}(G)$ and let A be a partial affine map. Fix $x \notin \text{Dom}(A)$ and let y, y' be points in the same orbit under $\text{Stab}_A(H)$. If there exists an extension $\hat{A}$ of A with $\hat{A}(x) = y$, then there also exists another extension $\tilde{A}$ of A with $\tilde{A}(x) = y'$.

Proof Since y and y' lie in the same $\text{Stab}_A(H)$ -orbit, there exists $g \in \text{Stab}_A(H)$ with $g(y) = y'$. Let $\hat{A}$ be an extension of A such that $\hat{A}(x) = y$, and define

$$\tilde{A} := g \circ \hat{A}.$$

Then $\tilde{A}$ is an affine map and $\tilde{A}(x) = g(\hat{A}(x)) = g(y) = y'$. Moreover, because $g \in \text{Stab}_A(H)$ fixes the points already prescribed by A , we have $\tilde{A}(u) = \hat{A}(u) = A(u)$ for all $u \in \text{Dom}(A)$, so $\tilde{A}$ is again an extension of A . $\square$

The contrapositive of this lemma says that if we branch with $A(x) = y$ and we don't find any extension there is no point in trying to branch with $A(x) = y'$ if y and y' are in the same $\text{Stab}_A(H)$ orbit. This can massively reduce branching.

5.1 Computation with groups

Now there are a going to consider how to compute various group operations in our algorithm. The group we are going to be working is the automorphism group. The first thing to consider is how to represent the group elements. The elements of our automorphism group are affine permutations $A : \mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m}$ where $A(\Gamma_F) \rightarrow \Gamma_F$. When we are going to do operations with this group we can treat this as a permutation on just the graph Γ_F which has size 2^n . So to store on of these permutation we just store a mapping $g : \Gamma_F \rightarrow \Gamma_F$

We have two different cases. In our automorphism algorithm we are finding the automorphisms as we are branching, so we need to update the group dynamically as we discover more automorphism elements, so that we can use them for pruning as soon as possible in the search. In our equivalence search the automorphism group is fixed so we just need to compute stabilizer subgroups for different branches.

We basically have 3 different operations we need to perform with the group H

- Compute the orbit partition of H .
- Update H when we find a new automorphism group.
- Compute $\text{Stab}_x(H)$ for some $x \in \mathbb{F}_2^{n+m}$.

This are all standard operations in computational group theory so we will not go into details here, we will do these operations using a Base and Strong Generating Set and Schreier–Sims for computing this [7]. We will briefly describe the computational complexities and how we perform these operations.

(1) Orbit partition. The problem is given some $H \leq \text{Aut}(G)$ partition the points into the orbit equivalence classes. We do this using a simple breath first search where we apply the generators of H to the visited points. The complexity is simply $\mathcal{O}(k \cdot N)$, where k is the number of generators of H and N is the size of the domain which is 2^{n+m} in our case. k is always and we use a strong generating set as the generators. In our algorithm we actually only do this breath first search inside a specific point cell.

(2) Updating the subgroup.

If our automorphism algorithm discovers a new CCZ/EA mapping $\tilde{A}$, we want to update the subgroups stored in the depth-first search tree. Let H be the current subgroup associated to a node with partial map A . We perform:

1. **Check that the group grows.** Test whether $\tilde{A} \notin \tilde{H}$ (equivalently, whether $\langle \tilde{H}, \tilde{A} \rangle$ is strictly larger than $\tilde{H}$). If not, there is nothing to update, and we don't continue with the 3 other steps.
2. **Augment the global group.** Add $\tilde{A}$ to the global subgroup $\tilde{H} \leq \text{Aut}(F)$ (maintained via a strong generating set).
3. **Recompute the node group.** Replace H by the stabilizer $H \leftarrow \text{Stab}_A(\tilde{H})$.
4. **Update the partition.** If H changed, recompute the orbit partition of the points.

Steps 3 and 4 are explained above/below. For Step 2 (augmenting the global group) we use the Schreier–Sims algorithm, which runs in polynomial time in the domain size of the permutations; in our case this domain is Γ_F/Γ_G with size 2^n . For step 1 this is the most important as we usually the new automorphism we discover does not enlarge the group and we stop here, but we can end up calling step 1 here quite a lot if the automorphism group is large. Luckily this is very efficient when we have a stabilizer chain.

(3) Stabilizers.

Computing the stabilizer is done in two different context, one is just computing $\text{Stab}_p(H)$ for a single point p , the other is to compute $\text{Stab}_A(H)$ for a partial map A . To compute $\text{Stab}_A(H)$ we only have to compute $\text{Stab}_{b_i}(H)$ for each a_i where $\langle a_1, \dots, a_k \rangle$. So the general problem we have to solve is to compute $\text{Stab}_{p_1, \dots, p_k}(H)$ for a set of points. We do this using what is called a stabilizer chain, this can also be done in polynomial time. We note that we do not need to recompute the stabilizer when a point cell gets individualized and we update A .

For the algorithm it turns out the group operations are generally not a big part of the runtime cost, so heavenly optimizing these operations is not necessary.

5.2 Quadratic Automorphisms

For quadratic function we get a set of automorphism for free, which allows us to fix $A(0, F(0))$.

Lemma 9 Let F be a quadratic function and define $\tau_a(x, y) : F_2^n \times F_2^m \rightarrow F_2^n \times F_2^m$ by

$$\tau_a(x, y) = (x + a, y + D_a F(x))$$

then $\tau_a \in \text{Aut}_{EA}(F)$ for all $a \in F_2^n$.

Proof Clearly τ_a is an affine map. To show that it is a permutation, define $\sigma_a(x, y) = (x + a, y + D_a F(x + a))$. then

$$\sigma_a(\tau_a(x, y)) = \sigma_a(x, y + D_a F(x) + D_a F(x + a))$$

and since $D_a F(x) + D_a F(x + a) = (F(x) + F(x + a)) + F(x) + F(x + a) = 0$ we get

$$= (x, y)$$

Now

$$\begin{aligned} \tau_a((x, F(x))) &= (x + a, F(x) + D_a F(x)) \\ &= (x + a, F(x) + F(x) + F(x + a)) \\ &= (x + a, F(x + a)) \end{aligned}$$

So $\tau_a(\Gamma_F) = \Gamma_F$. Also it is easy to see that τ_a is a EA mapping as the "input part" only depends on x . □

This lets us fix $A(0, F(0)) = (0, G(0))$ for quadratic functions.

Lemma 10 For quadratic functions F and G , $F \sim_{EA} G$ then there exist an A where $A(\Gamma_F) = \Gamma_G$ and $A((0, F(0))) = (0, G(0))$.

Proof Assume F and G quadratic with $A(\Gamma_F) = \Gamma_G$. By Lemma 9, $\tau_a \in \text{Aut}(G)$ for all $a \in F_2^n$. Assume $A(0, F(0)) = (p, G(p))$ for some p , then we get

$$\tau_p \circ A(0, F(0)) = \tau_p(p, G(p)) \tag{1}$$

$$= (p + p, G(p) + D_p G(p)) \tag{2}$$

$$= (0, G(p) + G(p + p) + G(p)) \tag{3}$$

$$= (0, G(0)) \tag{4}$$

So $\tau_p \circ A$ is the the required affine map. □

This lemma of course also holds for $\text{Aut}_{CCZ}(F)$ for quadratic functions.

5.3 Power maps

Many cryptographic important functions are power function or are CCZ equivalent with power functions. In this case we get some automorphisms for free [8] which can speed up the algorithm, we don't actually have to test whether or not the function is(or is equivalent) to a power map, we can just test whether or not the function have these automorphisms.

Multiplicative scaling.. Assume we represent $\mathbb{F}_{2^n}$ as an n -dimensional $\mathbb{F}_2$ -vector space and consider a power map $F(x) = x^d$. For any $a \in \mathbb{F}_{2^n}^\times$, the map

$$\mu_a : (x, y) \mapsto (ax, a^d y)$$

preserves the graph Γ_F since $(x, F(x)) \mapsto (ax, (ax)^d)$. This group $\{\mu_a\}_{a \in \mathbb{F}_{2^n}^\times}$ is cyclic and generated by μ_α whenever α is a primitive element of $\mathbb{F}_{2^n}^\times$.

Frobenius automorphisms.. Let $\varphi_k(x) = x^{2^k}$ be the 2^k -Frobenius on $\mathbb{F}_{2^n}$. For a power map $F(x) = x^d$ we have $F(\varphi_k(x)) = \varphi_k(F(x))$, hence

$$\phi_k : (x, y) \mapsto (x^{2^k}, y^{2^k})$$

also preserves Γ_F . This group $\{\phi_k\}_{k \in \{0, \dots, n-1\}}$ is also cyclic and generated by ϕ_1

These hold for any power function so we test whether or not μ_α and ϕ_1 are automorphism of the function of the function and add them to the automorphism group if they are. For the multiplicative scaling we of course do not know what d is but we can compute it the following way. If $f = x^d$ then $f(ax) = a^d f(x)$ then $a^d = \frac{f(x)}{f(ax)}$ if $f(ax) \neq 0$, we then just need to verify $f(ax) = a^d f(x)$ for all x .

Recently the automorphism groups of power functions have been completely classified [9], in addition to the 2 groups mentioned above, the automorphism groups of power function fall into one of 16 "exceptional" groups. seeding the equivalence algorithm with the known automorphisms can improve performance significantly.

6 The partial map

Though the algorithm we will maintain a partial affine map $A(x) = L(x) + t$. We will fix some anchor of A , denote this by $a = \text{anchor}(A)$, which is some fixed point of A , so $A(a) = b$ for some b . The linear part is encoded as $\mathcal{B}(A) = \{(u_1, v_1), \dots, (u_s, v_s)\}$ where $L(u_i) = v_i$. Let $\text{Dom}(A)$ denote the domain of A . If $a = \text{anchor}(A)$ is defined then we have $p \in \text{Dom}(A)$ if and only if $p - a \in \langle u_1, \dots, u_s \rangle$.

Algorithm 5: UPDATE(A, x, y) (incremental anchor + constraint-basis)

Input: Current DFS state A and a candidate pair $(x, y) \in \Gamma_F \times \Gamma_G$.
Output: **false** if inconsistent; otherwise updated state enforcing $A(x) = y$.
// A stores: (i) a two-way partial assignment on graph points,
 (ii) an anchor (a, b) with $b = A(a)$, and (iii) a list
 $\mathcal{B}(A) = ((u_1, v_1), \dots, (u_s, v_s))$ encoding constraints $L_A(u_i) = v_i$ for
 the linear part.
if $\text{anchor}(A)$ **is undefined** **then**
 | $\text{anchor}(A) \leftarrow (x, y)$;
 | **return** (*true*, A)
 $(a, b) \leftarrow \text{anchor}(A)$;
 $dx \leftarrow x + a$; $dy \leftarrow y + b$;
if $dx \in \langle u_1, \dots, u_s \rangle$ **then**
 | Let $c = (c_1, \dots, c_s)$ be dx written in the basis $u_1, \dots, u_s$;
 | $dy^* \leftarrow \sum c_i v_i$;
 | **if** $dy = dy^*$ **then**
 | | **return** (*true*, A)
 | **else**
 | | **return** *false*
else
 | **if** $dy \in \langle v_1, \dots, v_s \rangle$ **then**
 | | **return** *false*
Append (dx, dy) to $\mathcal{B}(A)$.
return (*true*, A);

We will have to verify that A is an valid CCZ/EA mapping. First for a CCZ mapping is that $A(\Gamma_F) = \Gamma_G$ and that A is a permutation. We denote the algorithm that verifies this as $\text{VALIDCCZ}(A)$. To verify that A is an EA mapping we of course need it to be an valid CCZ mapping first, in addition to this we need to be able to write A as

$$A = \begin{pmatrix} A_1 & 0 \\ A_3 & A_2 \end{pmatrix}$$

Where A_1, A_2 are affine permutations, and A_3 is affine. Let $\mathcal{B}(A) = (u_1, v_1), \dots, (u_s, v_s)$. To verify that a mapping is a valid EA mapping (A could be partial) we need to verify that

1. **Well-defined left part.** For each $(x_1, x_2), (x_1, x'_2) \in \text{Domain}(A)$ with

$$A(x_1, x_2) = (y_1, y_2), \quad A(x_1, x'_2) = (y'_1, y'_2),$$

verify that $y_1 = y'_1$.

2. **Injective left part.** For each $(x_1, x_2), (x'_1, x'_2) \in \text{Domain}(A)$ with $x_1 \neq x'_1$ and

$$A(x_1, x_2) = (y_1, y_2), \quad A(x'_1, x'_2) = (y'_1, y'_2),$$

verify that $y_1 \neq y'_1$.

3. **Closure of right part.** Let $Z = \{(0, z) : z \in \mathbb{F}_2^m\}$. An EA mapping must satisfy $L(Z) \subseteq Z$. To verify this, iterate over a basis element z of Z . If $z \in \langle u_1, \dots, u_s \rangle$, write $z = \sum_i c_i u_i$ and verify that $L(z) = \sum_i c_i v_i \in Z$.

Denote the algorithm that checks this as $\text{VALIDEA}(A)$.

7 Algorithms

Now we are ready to explain the complete algorithms for CCZ/EA equivalence/automorphism. The depth first search (DFS) algorithms is almost identical for the CCZ and EA algorithms so we will not repeat it, the only difference is that we need to check that A is a valid EA mapping before returning it.

7.1 Automorphism

We start with the automorphism algorithms. The difference from the equivalence algorithms is that we are discover the automorphisms during the algorithm so we need to update our data structures to reflect that. Also we don't necessary need to let the algorithm finish, as we might not need to to be sure we have found all automorphisms since a subgroup is still useful for the equivalence algorithms, also there is a very high likelihood that we have found generators for the entire group when we have found a few automorphisms. The depth first search of our automorphism algorithms look like this both for CCZ and EA.

Algorithm 6: DFS-AUTO($A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, H_{\text{stab}}, H_{\text{glob}}$)

Input: Partial affine map A , ordered partitions $\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$ (for the same function), a current stabilizer subgroup H_{stab} for orbit pruning, and a global accumulator $H_{\text{glob}} \leq \text{Aut}(F)$

Output: The updated global accumulator H_{glob}

(IsConsistent, $\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$) $\leftarrow$ REFINED($\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$);

if not IsConsistent **then**

return H_{glob} ;

if not VALIDCCZ(A) \setminus VALIDEA(A) **then**

return H_{glob} ;

if A is fully determined **then**

if A is a valid CCZ/EA mapping **then**

$H_{\text{glob}} \leftarrow \langle H_{\text{glob}}, A \rangle$;

return H_{glob} ;

Let P_i^F be the smallest cell in Π_F where $|P_i^F \setminus \text{Domain}(A)| > 1$;

$x \leftarrow$ any element of $P_i^F \setminus \text{Domain}(A)$;

$\mathcal{O} = (O_1, \dots, O_s) \leftarrow \text{ORBITPARTITION}(P_i^G, H_{\text{stab}})$;

$B \leftarrow \emptyset$; // touched branch points (previously chosen y 's)

$Y \leftarrow$ one element from each orbit $O_j \in \mathcal{O}$;

while $Y \neq \emptyset$ **do**

 Choose any $y \in Y$;

$B \leftarrow B \cup \{y\}$;

 (IsConsistent, A') $\leftarrow$ UPDATE(A, x, y);

if IsConsistent **then**

 Individualize x and y in Π_F and Π_G to get Π_F' and Π_G' ;

$H_{\text{glob}} \leftarrow \text{DFS-AUTO}(A', \Pi_F', \Pi_G', \Lambda_F, \Lambda_G, \text{Stab}_x(H_{\text{stab}}), H_{\text{glob}})$;

if $\text{Stab}_A(H_{\text{glob}}) > H_{\text{stab}}$ **then**

$H_{\text{stab}} \leftarrow \text{Stab}_A(H_{\text{glob}})$;

$\mathcal{O} = (O_1, \dots, O_s) \leftarrow \text{ORBITPARTITION}(P_i^G, H_{\text{stab}})$;

$Y \leftarrow$ one element from each orbit $O_j \in \mathcal{O}$ with $O_j \cap B = \emptyset$;

return H_{glob} ;

Remark 1 (Branching on orbit representatives) In the branching step (the **while**-loop), we only branch on orbit representatives in the current target cell. During the search the stabilizer subgroup may grow, so the orbit partition of the cell must be recomputed; however, we should not restart branching from scratch.

- B stores the branch points already tried (previously chosen y 's).
- Y is the queue of candidates to branch on next.

Initially, Y contains one element from each orbit. Each time we branch on some $y \in Y$, we add it to B . Whenever the stabilizer subgroup increases, we recompute the orbit partition and refill Y with one representative from each *new* orbit that does not intersect B .

The CCZ automorphism algorithm is fairly straightforward now, the only think we note is that we have found that is faster practically to discard hyperplanes immediately before even starting refinement algorithms. We do this by looking at the intersection count of each hyperplanes with the graph, make the initial cells based on that and then discard the largest cells (making sure to keep enough hyperplanes). This intersection count can easily is given by Walsh values and can be easily computed using FWHT.

Algorithm 7: CCZ-AUTOMORPHISM(F)

Input: Function $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$
Output: A generating set for a subgroup of $\text{Aut}_{\text{CCZ}}(F)$ found by the search
 $\Pi_F \leftarrow (\Gamma_F);$
 $\Pi_G \leftarrow (\Gamma_F);$
// Initialize and truncate the hyperplane partitions using
intersection numbers with Γ_F
Let $S_t := \{U \in \text{AGr}(n+m-1, n+m) : |U \cap \Gamma_F| = t\};$
Let $T := \{t : S_t \neq \emptyset\};$
Order T by decreasing $|2t - 2^n|$ (tie-break by increasing $|S_t|$);
 $\Lambda_F \leftarrow ();$
NumHyperplanes $\leftarrow 0;$
for $t \in T$ *while* NumHyperplanes $< 2^{n+1}$ **do**
 $\Lambda_F \leftarrow (\Lambda_F, S_t);$
 NumHyperplanes $\leftarrow \text{NumHyperplanes} + |S_t|;$
 $\Lambda_G \leftarrow \Lambda_F;$
 $A \leftarrow \text{partial map } \mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m};$
if F *is quadratic* **then**
 UPDATE($A, (0, F(0)), (0, F(0))$);
 $H \leftarrow \text{DFS}(A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, \emptyset);$
if F *is quadratic* **then**
 $\tau_a : (x, y) \mapsto (x + a, y + D_a F(x));$
 $H \leftarrow \langle H, \{\tau_a\}_{a \in \mathbb{F}_2^n} \rangle;$
return $H;$

The EA automorphisms algorithm is almost the same. The only difference is that we now partition the hyperplanes by both intersection count with the graph and by whether or not they are in $\mathcal{H}$.

Algorithm 8: EA-AUTOMORPHISM(F)

Input: Function $F : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$
Output: A generating set for a subgroup of $\text{Aut}_{EA}(F)$ found by the search
 $\Pi_F \leftarrow (\Gamma_F)$;
 $\Pi_G \leftarrow (\Gamma_F)$;
// Initialize the EA-invariant hyperplane partition (Lemma 1)
and truncate using intersection numbers with Γ_F
Let $H_{\alpha,c} := \{(x,y) : \alpha \cdot x = c\}$;
Let $\mathcal{H} := \{H_{\alpha,c} : \alpha \in \mathbb{F}_2^n \setminus \{0\}, c \in \mathbb{F}_2\}$;
 $\Lambda_F \leftarrow ()$;
 $\text{NumHyperplanes} \leftarrow 0$;
// Partition hyperplanes simultaneously by EA-type and
intersection size
Let $S_{t,1} := \{U \in \mathcal{H} : |U \cap \Gamma_F| = t\}$;
Let $S_{t,0} := \{U \in \text{AGr}(n+m-1, n+m) \setminus \mathcal{H} : |U \cap \Gamma_F| = t\}$;
Let $T := \{(t,c) : c \in \{0,1\} \wedge S_{t,c} \neq \emptyset\}$;
Order T by decreasing $|2t - 2^n|$ (tie-break by c decreasing, then by increasing
 $|S_{t,c}|$);
for $(t,c) \in T$ **while** $\text{NumHyperplanes} < 2^{n+1}$ **do**
 $\Lambda_F \leftarrow (\Lambda_F, S_{t,c})$;
 $\text{NumHyperplanes} \leftarrow \text{NumHyperplanes} + |S_{t,c}|$;
 $\Lambda_G \leftarrow \Lambda_F$;
 $A \leftarrow$ partial map $\mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m}$;
if F is quadratic **then**
 UPDATE($A, (0, F(0)), (0, F(0))$);
 $H \leftarrow \text{DFS}(A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, \emptyset)$;
if F is quadratic **then**
 $\tau_a : (x,y) \mapsto (x+a, y + D_a F(x))$;
 $H \leftarrow \langle H, \{\tau_a\}_{a \in \mathbb{F}_2^n} \rangle$;
return H ;

7.2 CCZ and EA equivalence

The DFS search for the equivalence algorithm are simpler since we do not have to update the orbit partitions the same way during the search.

Algorithm 9: DFS($A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, H$)

Input: Partial affine map A , ordered partitions $\Pi_F, \Pi_G, \Lambda_F, \Lambda_G$, and a current subgroup H for orbit pruning
Output: **true** iff an extension of A witnessing CCZ-equivalence is found (on this branch)
(IsConsistent, $\Pi_F, \Pi_G, \Lambda_F, \Lambda_G, A$) $\leftarrow$ REFINED($\Pi_F, \Pi_G, \Lambda_F, \Lambda_G, A$);
if *not* IsConsistent **then**
 return false;
if A is fully determined **then**
 return true;
 ; // For EA equivalence we must first check whether or not A is a valid EA mapping
Let P_i^F be the smallest cell in Π_F where $|P_i^F \setminus \text{Domain}(A)| > 1$;
 $x \leftarrow$ any element of $P_i^F \setminus \text{Domain}(A)$;
 $\mathcal{O} = (O_1, \dots, O_s) \leftarrow \text{ORBITPARTITION}(P_i^G, H)$;
for $O_j \in \mathcal{O}$ **do**
 $y \leftarrow$ any element of O_j ;
 (IsConsistent, A') $\leftarrow$ UPDATE(A, x, y);
 if IsConsistent **then**
 Found $\leftarrow$ DFS($A', \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, \text{Stab}_x(H)$);
 if Found **then**
 return true;
return false;

We now move the the equivalence algorithms. One of the optional parameters here is the subgroup of $\text{Aut}_{\text{CCZ}}(G)$. In the case where the function is highly symmetric (normal for cryptographically interesting functions) and are in-equivalent supplying this is very important and can turn infeasible cases to run in seconds. In our implementation we first let the automorphism algorithm run for a certain amount of seconds, and then set H to whatever we find. We have a time limit because there are some functions where our automorphisms algorithm to finish, but it still have found the entire group if we abort early. For one of the kasami APN functions for example the automorphism algorithm uses around 600 seconds to finish, but if we run the auto algorithm for just 10 second it does find the entire auto group. So in cases like this it makes sense to have a time limit since the full automorphism group is often discovered long before the automorphism algorithm finishes.

Algorithm 10: CCZEquivalence(F, G, H)

Input: Functions $F, G : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ and an optional pruning subgroup $H \leq \text{Aut}(G)$
Output: An affine permutation A with $A(\Gamma_G) = \Gamma_F$, or **false** if none exists
 $\Pi_F \leftarrow (\Gamma_F);$
 $\Pi_G \leftarrow (\Gamma_G);$
// Initialize and truncate the hyperplane partitions using intersection numbers (as in CCZ-AUTOMORPHISM)
Let $S_t^F := \{U \in \text{AGr}(n+m-1, n+m) : |U \cap \Gamma_F| = t\};$
Let $S_t^G := \{U \in \text{AGr}(n+m-1, n+m) : |U \cap \Gamma_G| = t\};$
// Abort early if the hyperplane ‘‘shape’’ differs
Let $N_t^F := |S_t^F|$ and $N_t^G := |S_t^G|;$
if $\exists t$ such that $N_t^F \neq N_t^G$ **then**
 return false;
Let $T := \{t : N_t^F > 0\};$
Order T by decreasing $|2t - 2^n|$ (tie-break by increasing $|S_t^F|$);
 $\Lambda_F \leftarrow ();$
 $\Lambda_G \leftarrow ();$
 $\text{NumHyperplanes} \leftarrow 0;$
for $t \in T$ **while** $\text{NumHyperplanes} < 2^{n+1}$ **do**
 $\Lambda_F \leftarrow (\Lambda_F, S_t^F);$
 $\Lambda_G \leftarrow (\Lambda_G, S_t^G);$
 $\text{NumHyperplanes} \leftarrow \text{NumHyperplanes} + |S_t^F|;$
 $A \leftarrow$ empty partial map, $\mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m};$
 $\text{IsEquivalent} \leftarrow \text{DFS}(A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, H);$
return IsEquivalent

Algorithm 11: EAEquivalence(F, G, H)

Input: Functions $F, G : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ (via their graphs) and an optional pruning subgroup $H \leq \text{Aut}(G)$

Output: An affine permutation A with $A(\Gamma_G) = \Gamma_F$, or **false** if none exists

$\Pi_F \leftarrow (\Gamma_F);$
 $\Pi_G \leftarrow (\Gamma_G);$
// Initialize and truncate the hyperplane partitions as in
EA-AUTOMORPHISM

Let $H_{\alpha,c} := \{(x, y) : \alpha \cdot x = c\};$
Let $\mathcal{H} := \{H_{\alpha,c} : \alpha \in \mathbb{F}_2^n \setminus \{0\}, c \in \mathbb{F}_2\};$
Let $S_{t,1}^F := \{U \in \mathcal{H} : |U \cap \Gamma_F| = t\};$
Let $S_{t,0}^F := \{U \in \text{AGr}(n+m-1, n+m) \setminus \mathcal{H} : |U \cap \Gamma_F| = t\};$
Let $S_{t,1}^G := \{U \in \mathcal{H} : |U \cap \Gamma_G| = t\};$
Let $S_{t,0}^G := \{U \in \text{AGr}(n+m-1, n+m) \setminus \mathcal{H} : |U \cap \Gamma_G| = t\};$
// Abort early if the hyperplane ‘‘shape’’ differs
Let $N_{t,c}^F := |S_{t,c}^F|$ and $N_{t,c}^G := |S_{t,c}^G|;$
if $\exists(t, c)$ such that $N_{t,c}^F \neq N_{t,c}^G$ **then**
 return false;

Let $T := \{(t, c) : c \in \{0, 1\} \wedge N_{t,c}^F > 0\};$
Order T by decreasing $|2t - 2^n|$ (tie-break by c decreasing, then by increasing $|S_{t,c}^F|$);
 $\Lambda_F \leftarrow ();$
 $\Lambda_G \leftarrow ();$
 $\text{NumHyperplanes} \leftarrow 0;$
for $(t, c) \in T$ while $\text{NumHyperplanes} < 2^{n+1}$ **do**
 $\Lambda_F \leftarrow (\Lambda_F, S_{t,c}^F);$
 $\Lambda_G \leftarrow (\Lambda_G, S_{t,c}^G);$
 $\text{NumHyperplanes} \leftarrow \text{NumHyperplanes} + |S_{t,c}^F|;$

$A \leftarrow$ empty partial map, $\mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m};$
if F is quadratic **then**
 UPDATE $(A, (0, F(0)), (0, G(0)));$

$\text{IsEquivalent} \leftarrow \text{DFS}(A, \Pi_F, \Pi_G, \Lambda_F, \Lambda_G, H);$
return IsEquivalent

Algorithm 12: CCZ-PARTITION($\mathcal{F}$)

Input: A list $\mathcal{F} = (F_1, \dots, F_N)$ of functions $F_i : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$
Output: A partition of $\mathcal{F}$ into CCZ-equivalence classes (represented as buckets)

```
 $\mathcal{B} \leftarrow ()$ ;  
; // list of buckets  
// Each bucket  $B \in \mathcal{B}$  stores a representative  $\text{rep}(B)$  and its  
// (full) CCZ automorphism group  $\text{Aut}(\text{rep}(B))$   
foreach  $F \in \mathcal{F}$  do  
     $H_F \leftarrow \text{CCZ-AUTOMORPHISM}(F)$ ;  
     $\sigma_F \leftarrow \text{GROUPSIGNATURE}(H_F)$ ;  
    ; // e.g., order + other cheap invariants  
     $\text{Placed} \leftarrow \text{false}$ ;  
    foreach  $B \in \mathcal{B}$  do  
        if  $\sigma_F = \text{GROUPSIGNATURE}(\text{Aut}(\text{rep}(B)))$  then  
            if  $\text{CCZEQUIVALENCE}(F, \text{rep}(B), \text{Aut}(\text{rep}(B)))$  then  
                Add  $F$  to bucket  $B$ ;  
                 $\text{Placed} \leftarrow \text{true}$ ;  
                break;  
    if not Placed then  
        Create new bucket  $B_{\text{new}}$  with  $\text{rep}(B_{\text{new}}) \leftarrow F$  and  
         $\text{Aut}(\text{rep}(B_{\text{new}})) \leftarrow H_F$ ;  
         $\mathcal{B} \leftarrow (\mathcal{B}, B_{\text{new}})$ ;  
return  $\mathcal{B}$ ;
```

We can of course do EA partitioning the same way.

References

- [1] Anne Canteaut and Léo Perrin. On ccz-equivalence, extended-affine equivalence, and function twisting. *Finite Fields and Their Applications*, 56:209–246, 2019.
- [2] Yves Edel and Alexander Pott. On the equivalence of nonlinear functions. In *Enhancing cryptographic primitives with techniques from error correcting codes*, pages 87–103. IOS Press, 2009.
- [3] Jeffrey Leon. Computing automorphism groups of error-correcting codes. *IEEE Transactions on Information Theory*, 28(3):496–511, 2003.
- [4] Anne Canteaut, Alain Couvreur, and Léo Perrin. Recovering or testing extended-affine equivalence. *IEEE Transactions on Information Theory*, 68(9):6187–6206, 2022.
- [5] Satoshi Yoshiara. Equivalences of quadratic apn functions. *Journal of Algebraic Combinatorics*, 35(3):461–475, 2012.
- [6] Boris Weisfeiler and Andrei Leman. The reduction of a graph to canonical form and the algebra which appears therein. *nti, Series*, 2(9):12–16, 1968.
- [7] Charles C Sims. Computational methods in the study of permutation groups. In *Computational problems in abstract algebra*, pages 169–183. Elsevier, 1970.
- [8] Ulrich Dempwolff. Ccz equivalence of power functions. *Designs, Codes and Cryptography*, 86(3):665–692, 2018.
- [9] Ulrich Dempwolff. Automorphism groups of power functions. *Journal of Group Theory*. doi: doi:10.1515/jgth-2023-0071. URL <https://doi.org/10.1515/jgth-2023-0071>.